Decode Ways II Explanation

Sure. This problem is **LeetCode 639 — Decode Ways II**. Your explanation has the right overall recursion + memoization idea. Let's organize it properly and make the logic clearer.

---

1. First understand: what is different from Decode Ways I?

In normal **Decode Ways**, the string contains digits:

```
"12"
```

We can decode it as:

```
1, 2  -> A B
12    -> L
```

So we have two choices at every index:

1. Take **one character**
2. Take **two characters**, if they form a number from `10` to `26`

New twist: `'*'`

Now the string can contain:

```
*
```

And `'*'` can represent any digit from:

```
1 to 9
```

For example:

```
"*"
```

can become:

```
1, 2, 3, 4, 5, 6, 7, 8, 9
```

So it has **9 possibilities**.

---

2. Recursive state

We define:

```
C++
decodings(ind)
```

Meaning:

> How many valid ways are there to decode the string starting from index `ind`?

For example:

```
s = "1*2"
     ↑
    ind
```

`decodings(ind)` will try all valid possibilities starting from this position.

---

3. Base cases

Case 1: We reached the end

```
C++
if (ind == s.size())
```

```
    return 1;
```

Why return `1`?

Because reaching the end means:

> We successfully decoded the complete string.

For example:

```
"12"

Take "1"
   |
Take "2"
   |
Reached end
   |
return 1
```

That one valid path contributes `1` decoding.

Case 2: Current character is `'0'`

```
C++
if (s[ind] == '0')
    return 0;
```

Because `0` cannot be decoded alone.

For example:

```
"06"
```

At index `0`:

```
0 ❌
```

Invalid.

But remember:

```
10 ✅
20 ✅
```

are valid because `0` is combined with the previous digit.

---

4. Choice 1 — Take one character

Normally:

```
C++
long long oneCharChoice = decodings(ind + 1, s, dp);
```

This means:

> Decode the current character as one letter, then recursively solve the remaining string.

If current character is a digit

Example:

```
s = "123"

ind = 0

Take:
"1"

Then solve:
"23"
```

So:

```
oneCharChoice = decodings(ind + 1)
```

If current character is `'*'`

Example:

```
"*"
```

It can become:

```
1
2
3
4
5
6
7
8
9
```

So there are:

```
9 choices
```

Therefore:

```
C++
if (s[ind] == '*')
    oneCharChoice = oneCharChoice * 9;
```

So conceptually:

```
ways = 9 × ways(ind + 1)
```

5. Choice 2 — Take two characters

Now we look at:

```
s[ind] and s[ind + 1]
```

We need to count how many ways these two characters can form a number between:

```
10 to 26
```

Your code separates this into two cases.

CASE A: First character is `'1'` or `'*'`

```
C++
if (s[ind] == '1' || s[ind] == '*')
```

Why?

Because numbers starting with **1** can be:

```
10
11
12
13
14
15
16
17
18
19
```

All of them are valid.

Example: `"1*"`

```
1*
```

* can be:

```
1 to 9
```

Therefore:

```
11
12
13
14
15
16
17
18
19
```

All 9 are valid.

So:

```C++
if (s[ind + 1] == '*')
    twoCharChoice1 *= 9;
```

Example: `"12"`

Only one possibility:

```
12
```

So:

```C++
twoCharChoice1 = decodings(ind + 2, s, dp);
```

No multiplication is needed.

CASE B: First character is `'2'` or `'*'`

```C++
if (s[ind] == '2' || s[ind] == '*')
```

Numbers starting with **2** are more restricted.

Valid numbers are:

```
20
21
22
23
24
25
26
```

So the next digit must be:

```
0 to 6
```

Example: `"2*"`

`*` can become:

```
1
2
3
4
5
6
```

Thus valid pairs are:

```
21
22
23
24
25
26
```

There are:

```
6 choices
```

Therefore:

```
C++
twoCharChoice2 = 6 * decodings(ind + 2);
```

Example: "23"

Since:

```
3 <= 6
```

23 is valid.

So:

```
C++
twoCharChoice2 = decodings(ind + 2);
```

Example: "27"

Here:

```
7 > 6
```

So:

```
27 ❌
```

No two-character decoding.

---

6. Important case: when first character itself is `'*'`

This is where your code's separation is useful.

Suppose:

```
"**"
```

The first * can behave as:

As 1

```
11, 12, 13, ..., 19
```

→ **9** possibilities

As 2

```
21, 22, 23, 24, 25, 26
```

→ **6** possibilities

Therefore two-character decoding gives:

```
9 + 6 = 15 possibilities
```

This is exactly why your code has:

```
C++
twoCharChoice1
```

and:

```
C++
twoCharChoice2
```

separately.

---

7. Recursion tree intuition

Suppose:

```
s = "1*"
```

At index **0**:

```
                  "1*"
                    |
          -------------------------
          |                       |
      Take "1"                Take "1*"
          |                       |
         "*"                     end
          |                       |
    ----------------              1
    1 2 3 4 5 6 7 8 9
    |
    9 ways
```

Total:

```
Single character route = 9
Double character route = 9

Answer = 18
```

8. Memoization

Without memoization, we may calculate:

```
C++
decodings(ind + 2)
```

multiple times.

So:

```
C++
dp[ind]
```

stores:

> Number of ways to decode the substring starting from index **ind**.

For example:

```
dp[3]
```

means:

```
Number of decoding ways for:

s[3 ... n-1]
```

Small correction to your explanation: it is **not** the number of decodings "till ith index"; rather, because the recursion moves forward, **dp[i]** represents the number of ways to decode the **suffix starting at index i**.

Proper formatted code

```
C++
class Solution {
```

```
            // ------------------------------------------------
            // Case 1: First character behaves as '1'
            // Valid pairs: 10 to 19
            // ------------------------------------------------

            if (s[ind] == '1' || s[ind] == '*') {

                twoCharChoice1 = decodings(ind + 2, s, dp);

                // "1*" or "*"
                // Second '*' can be 1 to 9
                if (s[ind + 1] == '*') {
                    twoCharChoice1 =
                        (twoCharChoice1 * 9) % MOD;
                }
            }


            // ------------------------------------------------
            // Case 2: First character behaves as '2'
            // Valid pairs: 20 to 26
            // ------------------------------------------------

            if (s[ind] == '2' || s[ind] == '*') {

                // Example: "2*" or "**"
                if (s[ind + 1] == '*') {

                    twoCharChoice2 =
                        decodings(ind + 2, s, dp);

                    // '*' can be 1 to 6
                    twoCharChoice2 =
                        (twoCharChoice2 * 6) % MOD;
                }

                // Example: 20, 21, 22 ... 26
                else if (s[ind + 1] <= '6') {

                    twoCharChoice2 =
                        decodings(ind + 2, s, dp);
                }
            }
        }


        // Add all valid possibilities
        return dp[ind] =
            (oneCharChoice +
             twoCharChoice1 +
             twoCharChoice2) % MOD;
    }


    int numDecodings(string s) {

        int n = s.size();

        // dp[i] = ways to decode substring starting from i
        vector<int> dp(n, -1);

        return decodings(0, s, dp);
    }
};
```

One small note about the `'*'` definition

In this problem, `'*'` represents digits **1 through 9**, not `0 through 9`. The reason `20` can still be valid is when the second character is literally `'0'`, for example `"20"`.

Final recurrence

At every index:

```
ways(ind)
=
single-character ways
+
two-character ways where first behaves as 1
+
two-character ways where first behaves as 2
```

Or:

```
dp[ind] =
oneCharChoice
+
twoCharChoice1
+
twoCharChoice2
```

all modulo `1e9 + 7`.

Time Complexity: `O(n)`
Space Complexity: `O(n)` for DP + `O(n)` recursion stack.

Yes. Let's convert **your memoization code** → **tabulation** step by step.

The main thing to understand is:

> **Memoization:** start from `0` and recursively go forward.
> **Tabulation:** start from the end and fill answers backward.

Step 1: Look at what memoization state means

In your recursion:

```
C++
decodings(ind, s, dp)
```

means:

> Number of ways to decode the string starting from index `ind`.

So:

```
dp[ind] = number of ways to decode s[ind ... n-1]
```

For example:

```
s = "1*2"

index:   0   1   2
         1   *   2
```

Then:

```
dp[0] = ways to decode "1*2"
dp[1] = ways to decode "*2"
dp[2] = ways to decode "2"
dp[3] = ways to decode ""
```

Notice:

```
dp[ind] depends on dp[ind + 1]
                    dp[ind + 2]
```

So in tabulation, if we need future indexes:

```
dp[i + 1]
dp[i + 2]
```

we should fill the DP array from:

```
right → left
```

Step 2: Convert the base case

In memoization:

```
C++
```

```
if (ind == s.size()) {
    return 1;
}
```

This means:

```
dp[n] = 1
```

Why?

Because an empty string after successfully consuming everything represents **one valid decoding path**.

So:

```
C++
vector<long long> dp(n + 1, 0);

dp[n] = 1;
```

Now our DP initially looks like:

```
s = "1*2"

index:      0     1     2     3
            1     *     2    END

dp:         ?     ?     ?      1
```

Step 3: Convert `'0'` base case

In recursion:

```
C++
if (s[ind] == '0') {
    return 0;
}
```

In tabulation:

```
C++
if (s[ind] == '0') {
    dp[ind] = 0;
    continue;
}
```

Because `'0'` cannot be decoded as a single character.

Step 4: Convert the single-character choice

Memoization:

```
C++
long long oneCharChoice = decodings(ind + 1, s, dp);

if (s[ind] == '*') {
    oneCharChoice = (oneCharChoice * 9) % MOD;
}
```

Replace:

```
decodings(ind + 1)
```

with:

```
dp[ind + 1]
```

Therefore:

```
C++
long long oneCharChoice = dp[ind + 1];

if (s[ind] == '*') {
    oneCharChoice = (oneCharChoice * 9) % MOD;
}
```

Step 5: Convert the first two-character case

Memoization:

```
C++
if (s[ind] == '1' || s[ind] == '*') {

    twoCharChoice1 = decodings(ind + 2, s, dp);

    if (s[ind + 1] == '*') {
        twoCharChoice1 =
            (twoCharChoice1 * 9) % MOD;
    }
}
```

Simply replace:

```
decodings(ind + 2)
```

with:

```
dp[ind + 2]
```

So:

```
C++
if (s[ind] == '1' || s[ind] == '*') {

    twoCharChoice1 = dp[ind + 2];

    if (s[ind + 1] == '*') {
        twoCharChoice1 =
            (twoCharChoice1 * 9) % MOD;
    }
}
```

Step 6: Convert the second two-character case

Memoization:

```
C++
if (s[ind] == '2' || s[ind] == '*') {

    if (s[ind + 1] == '*') {

        twoCharChoice2 =
            decodings(ind + 2, s, dp);

        twoCharChoice2 =
            (twoCharChoice2 * 6) % MOD;
    }

    else if (s[ind + 1] <= '6') {

        twoCharChoice2 =
            decodings(ind + 2, s, dp);
    }
}
```

Again:

```
decodings(ind + 2)
```

becomes:

```
dp[ind + 2]
```

Therefore:

```
C++
if (s[ind] == '2' || s[ind] == '*') {

    if (s[ind + 1] == '*') {

        twoCharChoice2 = dp[ind + 2];

        twoCharChoice2 =
            (twoCharChoice2 * 6) % MOD;
    }

    else if (s[ind + 1] <= '6') {

        twoCharChoice2 = dp[ind + 2];
    }
}
```

Step 7: Where does the loop start?

Our recurrence needs:

```
dp[ind + 1]
dp[ind + 2]
```

Therefore, start from the last character:

```
n - 1
```

and go backwards:

```
C++
for (int ind = n - 1; ind >= 0; ind--)
```

Complete conversion

Memoization

```
decodings(ind)
        ├── decodings(ind + 1)
        └── decodings(ind + 2)
```

Tabulation

```
dp[n] = 1

for ind = n - 1 to 0:

    dp[ind]
        uses
        ├── dp[ind + 1]
        └── dp[ind + 2]
```

So the recursion tree has now become a backward loop.

Tabulation code

```
C++
class Solution {
public:
    const int MOD = 1e9 + 7;

    int numDecodings(string s) {

        int n = s.size();

        // dp[i] = number of ways to decode
        // the substring starting from index i
        vector<long long> dp(n + 1, 0);

        // Base case:
        // Reaching after the last character means
        // one valid decoding is completed
        dp[n] = 1;


        // Fill dp from right to left
        for (int ind = n - 1; ind >= 0; ind--) {

            // '0' cannot be decoded alone
            if (s[ind] == '0') {
                dp[ind] = 0;
                continue;
            }


            // ============================================
            // Choice 1: Take one character
            // ============================================

            long long oneCharChoice = dp[ind + 1];

            // '*' can represent 1 to 9
            if (s[ind] == '*') {
                oneCharChoice =
                    (oneCharChoice * 9) % MOD;
            }


            // ============================================
            // Choice 2: Take two characters
            // ============================================

            long long twoCharChoice1 = 0;
            long long twoCharChoice2 = 0;


            // We need a next character
            if (ind + 1 < n) {
```

```
            // ----------------------------------------
            // Case 1:
            // First character behaves as '1'
            // Valid range: 10 to 19
            // ----------------------------------------

            if (s[ind] == '1' || s[ind] == '*') {

                twoCharChoice1 = dp[ind + 2];

                // Example: "1*" or "**"
                // Second '*' can be 1 to 9
                if (s[ind + 1] == '*') {

                    twoCharChoice1 =
                        (twoCharChoice1 * 9) % MOD;
                }
            }


            // ----------------------------------------
            // Case 2:
            // First character behaves as '2'
            // Valid range: 20 to 26
            // ----------------------------------------

            if (s[ind] == '2' || s[ind] == '*') {

                // Example: "2*" or "**"
                if (s[ind + 1] == '*') {
```

Let's dry run the tabulation for `"1*"`

```
s = "1*"
n = 2
```

Initially:

```
index:      0      1      2
            1      *     END

dp:         0      0      1
```

We start from:

```
ind = 1
```

`ind = 1`, character = `'*'`

Single character

`*` can be:

```
1 2 3 4 5 6 7 8 9
```

So:

```
oneCharChoice = dp[2] × 9
              = 1 × 9
              = 9
```

There is no next character, so no two-character choice.

Therefore:

```
dp[1] = 9
```

Now:

```
index:      0      1      2
            1      *     END

dp:         ?      9      1
```

`ind = 0`, character = `'1'`

Choice 1: Take `"1"`

```
oneCharChoice = dp[1]
              = 9
```

This represents:

```
1 | *
```

The `*` has 9 possibilities.

Choice 2: Take `"1*"`

Since first character is `'1'`:

```
10 to 19
```

But here `*` represents **`1`** to **`9`**, according to the problem's wildcard definition, so:

```
11, 12, 13, ..., 19
```

Thus:

```
twoCharChoice1 = dp[2] × 9
               = 1 × 9
               = 9
```

No second choice involving `'2'`.

So:

```
dp[0] = 9 + 9
      = 18
```

Final DP:

```
index:      0      1      2
            1      *     END

dp:        18      9      1
```

Therefore:

```
Answer = dp[0] = 18
```

The main conversion trick to remember

Whenever converting this memoization:

```
C++
helper(ind)
```

to tabulation, do this:

```
helper(n)  → dp[n]
helper(i+1) → dp[i+1]
```

```
helper(i+2) → dp[i+2]
```

Since **`dp[i]`** depends on indexes **ahead of it**, fill the array:

```
n - 1 → 0
```

That is the entire memoization-to-tabulation conversion for this problem.